

Université de Paris Cité

Rapport du Projet

Data Quality

Réalisé par :

MERABTI Amar

HAMMOUCHE Lynda

2025–2026

Table des matières

1	Introduction	3
2	Présentation du contexte et des scénarios métier	3
2.1	Scénario 1 : Driver – Analyse de Performance et Optimisation des Revenus	3
2.1.1	Contexte	3
2.1.2	Objectifs métier	3
2.1.3	Questions métier typiques	3
2.1.4	Utilisateurs	3
2.1.5	Colonnes critiques et pondération	4
2.2	Scénario 2 : Passager – Estimation de Coût et Planification de Trajet . . .	4
2.2.1	Contexte	4
2.2.2	Objectifs métier	4
2.2.3	Questions métier typiques	4
2.2.4	Utilisateurs	4
2.2.5	Colonnes critiques et pondération	5
2.3	Analyse des différences entre les scénarios	5
2.3.1	Priorités divergentes	5
2.3.2	Impact sur l'évaluation qualité	5
3	Injection d'erreurs de qualité	5
3.1	Valeurs manquantes (Completeness)	6
3.2	Doublons (Unicité)	6
3.3	Valeurs incorrectes (Accuracy)	6
3.4	Injection d'incohérences temporelles (Consistency)	6
3.5	Incohérences géographiques (Consistency)	6
4	Data Profiling	7
5	Data Quality Assessment	7
5.1	Les Dimensions	7
5.1.1	Dimension 1 : Completeness	7
5.1.2	Dimension 2 : Accuracy	8
5.1.3	Dimension 3 : Consistency	8
5.1.4	Dimension 4 : Unicité (Uniqueness)	9
5.2	Assessment	10
5.3	Vulnérabilités spécifiques par scénario	10
5.3.1	Scénario Driver	10
5.3.2	Scénario Passenger	10
5.3.3	Vulnérabilités communes	11
5.3.4	Problème d'unicité détecté	11
5.3.5	Note méthodologique sur la cohérence	11
5.4	Conclusion	11
6	Data Quality Improvement	12
6.1	Stratégie Globale de Correction	12
6.1.1	Phase 1 : Normalisation Syntaxique	12
6.1.2	Phase 2 : Déduplication	12

6.1.3	Phase 3 : Imputation des Valeurs Manquantes	12
6.1.4	Phase 4 : Validation de Cohérence	12
6.2	Corrections Mises en Œuvre	13
6.2.1	Normalisation des Formats de Dates	13
6.2.2	Normalisation des Montants Monétaires	13
6.2.3	Normalisation des Distances	13
6.2.4	Élimination des Doublons	13
6.2.5	Imputation des Valeurs Manquantes	13
6.2.6	Correction des Coordonnées GPS Erronées	14
6.2.7	Correction des Incohérences Temporelles	15
6.2.8	Imputation des Attributs Catégoriels et Discrets	15
6.3	Résultat de la Correction	16
7	Échange Inter-Groupes	16
7.1	Résultat de l'Assessment	17
7.2	Résultat de l'Improvement	17
7.3	Erreurs Non Détectées	17
7.4	Limites du Pipeline de Detection et de Correction	18
8	Conclusion	19

1 Introduction

La qualité des données constitue aujourd’hui un enjeu stratégique majeur, des données dégradées pouvant conduire à des décisions erronées, des pertes financières et une perte de confiance des utilisateurs. Dans le secteur du transport urbain, et en particulier des services de taxi, la fiabilité des données est essentielle, car elle influence directement la rentabilité des chauffeurs, la satisfaction des passagers et l’efficacité opérationnelle.

Ce projet vise à évaluer et améliorer la qualité d’un jeu de données de courses de taxis new-yorkais collectées en février 2023, en combinant l’injection d’erreurs réalistes, une évaluation multicritère de la qualité et des stratégies de correction adaptées au contexte métier.

L’objectif est de montrer qu’une approche rigoureuse de Data Quality Management permet de garantir la *fitness for use* des données, en analysant l’impact des défauts de qualité selon deux perspectives métier distinctes : celle du chauffeur et celle du passager.

2 Présentation du contexte et des scénarios métier

Pour évaluer le *fitness for use* du dataset, nous avons défini deux scénarios représentant des cas d’usage distincts avec des exigences de qualité différentes.

2.1 Scénario 1 : Driver – Analyse de Performance et Optimisation des Revenus

2.1.1 Contexte

Un chauffeur de taxi ou un gestionnaire de flotte analyse les données historiques pour identifier les stratégies permettant de maximiser les revenus quotidiens.

2.1.2 Objectifs métier

- Identifier les zones géographiques et horaires à forte rentabilité
- Calculer les revenus moyens par heure, par zone
- Analyser l’efficacité opérationnelle (ratio distance/tarif, durée moyenne des courses)
- Planifier les shifts de travail de manière optimale

2.1.3 Questions métier typiques

- Quels sont les créneaux horaires les plus rentables ?
- Quelles zones géographiques concentrent le plus (ou le moins) de courses ? (Afin d’adapter la répartition des chauffeurs)

2.1.4 Utilisateurs

Chauffeurs indépendants, gestionnaires de flotte, analystes business.

2.1.5 Colonnes critiques et pondération

Colonne	Poids (1-10)	Justification
fare_amount	10	Métrique centrale pour le calcul des revenus (reste l'objectif prioritaire).
Coordonnées GPS	9	Critique. Indispensable pour identifier les zones de concentration des courses et effectuer la jointure avec les quartiers.
pickup_datetime	8	Essentiel pour identifier les patterns temporels et planifier les shifts.
dropoff_datetime	6	Permet de calculer la durée des courses (efficacité) et la rotation des véhicules.
trip_distance_miles	5	Utile pour calculer le ratio de rentabilité (revenu/mile) et l'usure du véhicule.
passenger_count	1	Information secondaire pour ce scénario.
payment_type	1	Information secondaire.

TABLE 1 – Pondération des colonnes critiques pour le Scénario 1

2.2 Scénario 2 : Passager – Estimation de Coût et Planification de Trajet

2.2.1 Contexte

Un utilisateur d'application de mobilité (touriste ou résident) souhaite estimer le coût et la durée d'un trajet avant de commander un taxi. Il cherche la transparence des prix et la prévisibilité.

2.2.2 Objectifs métier

- Fournir une estimation précise du tarif (*Fare Estimation*) basée sur le trajet prévu
- Estimer la durée du voyage (ETA) en fonction de l'heure et du trafic historique
- Comparer le coût du taxi par rapport aux alternatives (VTC, transports en commun)
- Identifier les périodes « creuses » pour voyager moins cher

2.2.3 Questions métier typiques

- Quel est le budget à prévoir pour un trajet de A à B (ex : JFK vers Manhattan) ?
- Combien de temps durera la course si je pars à 18h00 vs 20h00 ?
- Y a-t-il des surcharges tarifaires récurrentes sur cet itinéraire ?

2.2.4 Utilisateurs

Voyageurs, applications de comparaison de transport, touristes.

Colonne	Poids (1-10)	Justification
trip_distance_miles	10	Critique. C'est le facteur principal de la tarification au compteur. Une erreur ici fausse totalement l'estimation pour le client.
fare_amount	10	La donnée cible. Indispensable pour entraîner des modèles de prédiction de prix fiables.
dropoff_datetime	9	Crucial pour calculer la durée du trajet (Pickup – Dropoff). Le client veut savoir « combien de temps ça prend ».
pickup_datetime	8	Nécessaire pour gérer la saisonnalité (heure de pointe, tarif de nuit, jour férié).
Coordonnées GPS	7	Important pour définir les points A (Départ) et B (Arrivée) et appliquer les forfaits spécifiques (ex : aéroports).
passenger_count	1	Information secondaire.
payment_type	1	Information secondaire.

TABLE 2 – Pondération des colonnes critiques pour le Scénario 2

2.2.5 Colonnes critiques et pondération

2.3 Analyse des différences entre les scénarios

Les deux scénarios illustrent comment les mêmes données peuvent avoir des exigences de qualité différentes selon le cas d'usage :

2.3.1 Priorités divergentes

- Le scénario Driver privilégie les coordonnées GPS (poids 9) pour l'analyse géographique, tandis que le scénario Passenger privilégie la distance et la durée (poids 10 et 9) pour l'estimation tarifaire.
- Pour le Driver, `pickup_datetime` (poids 8) est crucial pour identifier les patterns temporels rentables. Pour le Passenger, `dropoff_datetime` (poids 9) est plus important car il détermine la durée du trajet.

2.3.2 Impact sur l'évaluation qualité

- Une distance manquante (`trip_distance_miles`) aura un impact **CRITIQUE** pour le Passenger (poids 10) mais **MOYEN** pour le Driver (poids 5).
- Des coordonnées GPS erronées seront bloquantes pour le Driver (poids 9) mais moins critiques pour le Passenger (poids 7) qui se concentre sur la distance totale.

Cette pondération différenciée permettra de calculer des scores de qualité contextualisés et d'évaluer le *fitness for use* de manière nuancée selon chaque perspective métier.

3 Injection d'erreurs de qualité

Afin de simuler des problèmes de qualité rencontrés en conditions réelles, plusieurs types d'erreurs ont été injectés artificiellement dans le jeu de données.

3.1 Valeurs manquantes (Complétude)

Des valeurs manquantes ont été introduites dans les variables `fare_amount` et `trip_distance_miles` (20 observations chacune), par sélection aléatoire. Cette approche reproduit des pertes d'information pouvant survenir lors de la collecte, de la transmission ou du stockage des données.

3.2 Doublons (Unicité)

Des doublons ont été créés en sélectionnant aléatoirement 20 lignes, puis en les ré-injectant deux à trois fois dans le dataset, avant mélange. Cette manipulation simule des erreurs courantes telles que des enregistrements multiples d'une même course lors de processus d'ingestion ou de synchronisation.

3.3 Valeurs incorrectes (Accuracy)

Des valeurs irréalistes ont été introduites dans `fare_amount` et `trip_distance_miles` sous forme de montants et distances négatifs. Ces anomalies reflètent des erreurs de saisie, de capteurs ou de traitements automatiques non contrôlés, entraînant des données non conformes aux règles métier.

3.4 Injection d'incohérences temporelles (Consistency)

Afin de simuler des problèmes de cohérence temporelle (*Consistency*), certaines valeurs de `pickup_datetime` ont été volontairement modifiées. Treize lignes ont été sélectionnées aléatoirement et leur date de prise en charge a été remplacée par une date correspondant à l'année 2025, alors que le reste du dataset concerne l'année 2023. Cette altération reproduit des erreurs réalistes liées à des problèmes de synchronisation, de fuseaux horaires ou de mises à jour incorrectes, et génère des incohérences rendant les analyses temporelles et historiques non fiables.

3.5 Incohérences géographiques (Consistency)

Afin de simuler des défaillances de géolocalisation, des coordonnées GPS aberrantes ont été introduites. Ces erreurs correspondent à des points situés sur l'eau ou en dehors de la zone d'activité normale des taxis new-yorkais.

L'analyse exploratoire a toutefois révélé que le dataset contenait déjà ce type d'erreurs de manière naturelle. Nous avons donc adopté une approche mixte en conservant ces incohérences préexistantes tout en ajoutant des injections contrôlées. Cette combinaison permet de capturer à la fois des erreurs réelles (perte de signal GPS, mauvaise calibration des capteurs) et des anomalies artificielles standardisées.

Concrètement, 6 incohérences géographiques supplémentaires ont été injectées : 3 affectant les coordonnées de départ (`pickup_latitude`, `pickup_longitude`) et 3 affectant les coordonnées d'arrivée (`dropoff_latitude`, `dropoff_longitude`). Ces points ont été positionnés sur des zones aquatiques ou à des distances significatives de la zone de service habituelle.

4 Data Profiling

Le data profiling révèle un jeu de données relativement propre mais présentant quelques anomalies clés. Le dataset comprend 10 variables et 848 observations, avec 43 cellules manquantes (0,5%) et 20 lignes dupliquées (2,4%). Les types de données se répartissent en 2 DateTime, 6 numériques et 2 catégorielles.

La variable `trip_distance_miles` (341 valeurs distinctes) présente 20 valeurs manquantes (2,4%) et quelques distances négatives (minimum = -4.83 miles). La variable `fare_amount` compte 23 valeurs manquantes (2,7%) et des valeurs négatives (minimum = -24.76 dollars). Les moyennes s'établissent respectivement à environ 2.51 miles et 12.30 dollars.

Les coordonnées géographiques montrent des écarts et valeurs aberrantes notables. Les latitudes `pickup_latitude` et `dropoff_latitude` couvrent généralement 40.60 à 40.92 degrés, mais `dropoff_latitude` affiche un minimum très bas à 36.35. Les longitudes sont majoritairement négatives autour de -73.8, avec `dropoff_longitude` présentant une forte asymétrie et une valeur maximale aberrante à 4.29, indiquant des positions géographiques manifestement erronées.

Les corrélations importantes sont conformes aux attentes : forte corrélation entre `trip_distance_miles` et `fare_amount` (environ 0.78) et entre coordonnées pickup/dropoff, validant l'usage de ces colonnes pour des corrections géospatiales et monétaires. Pour les variables catégorielles, `payment_type` est dominé par les cartes bancaires (568) puis les espèces (198), tandis que `passenger_count` est bien réparti entre 1 et 5 passagers. La distribution révèle des asymétries élevées sur certains champs et des duplicatas fréquents (jusqu'à 4 occurrences).

5 Data Quality Assessment

5.1 Les Dimensions

5.1.1 Dimension 1 : Completeness

La complétude mesure le degré de présence des données, c'est-à-dire la proportion de valeurs effectivement renseignées par rapport au nombre total de valeurs attendues. Une donnée manquante peut empêcher certaines analyses ou fausser les résultats, en particulier lorsque la variable concernée est critique pour le scénario métier étudié.

Pour une colonne donnée, le score de complétude est calculé comme suit :

- Les valeurs manquantes sont identifiées via `isnull()`
- Le ratio de valeurs valides est calculé :

$$\text{Complétude brute} = \frac{\text{Nombre de valeurs non manquantes}}{\text{Nombre total de valeurs}} \quad (1)$$

- Ce ratio est ensuite pondéré par l'importance métier de la colonne à l'aide d'une élévation à la puissance du poids associé au scénario :

$$\text{completeness_score} = \left(\frac{\text{nombre_valeurs_présentes}}{\text{nombre_total_lignes}} \right)^{\text{poids}} \quad (2)$$

5.1.2 Dimension 2 : Accuracy

La dimension d'exactitude (*Accuracy*) mesure dans quelle mesure les valeurs présentes dans le jeu de données sont correctes, plausibles et conformes aux règles métier attendues.

Règles de validation métier Des règles de validation spécifiques ont été définies pour chaque variable critique, en cohérence avec le contexte des courses de taxis new-yorkais :

- **fare_amount** : valeur numérique comprise entre 2.50\$ (tarif minimum réglementaire) et 500\$, afin d'éliminer les montants aberrants ou irréalistes.
- **trip_distance_miles** : distance strictement positive et inférieure ou égale à 100 miles, couvrant l'ensemble des trajets plausibles dans la zone d'activité.
- **passenger_count** : nombre de passagers compris entre 1 et 5, correspondant aux capacités standards des taxis.
- Variables temporelles (**pickup_datetime**, **dropoff_datetime**) : dates valides et comprises entre 2020 et 2024, garantissant une cohérence temporelle avec la période d'étude.
- Coordonnées GPS (pickup et dropoff) : les paires latitude/longitude doivent correspondre à des localisations valides à New York et ne pas se situer sur des zones non exploitables (fleuves, lacs, zones hors périmètre). Cette vérification repose sur une fonction de validation géographique dédiée.

Calcul du score d'exactitude Le score d'exactitude pour une colonne donnée est calculé selon la formule suivante :

$$\text{Score}_{\text{accuracy}} = \left(\frac{n_{\text{valides}}}{n_{\text{non-null}}} \right)^{\text{poids}} \quad (3)$$

où n_{valides} représente le nombre de valeurs respectant les règles métier, $n_{\text{non-null}}$ le nombre de valeurs non manquantes.

5.1.3 Dimension 3 : Consistency

La cohérence (*Consistency*) évalue le respect des relations logiques et métier entre plusieurs attributs d'un même enregistrement. Contrairement à l'exactitude (*accuracy*), qui vérifie la validité d'une valeur prise isolément, la cohérence s'intéresse à la compatibilité des données entre elles.

Comme pour les autres dimensions, l'évaluation de la cohérence est contextualisée par scénario métier (Driver ou Passenger) grâce à une pondération reflétant l'importance relative de chaque colonne.

Règles de cohérence inter-attributs Plusieurs règles de cohérence métier ont été définies :

Type de cohérence	Règle de validation
Cohérence temporelle	<code>pickup_datetime</code> doit être strictement antérieur à <code>dropoff_datetime</code> . Cette règle garantit qu’une course ne se termine pas avant d’avoir commencé, condition indispensable pour toute analyse de durée ou de planification.
Cohérence tarif – distance	Pour une distance strictement positive (<code>trip_distance_miles</code> > 0), le ratio tarif/distance doit rester dans une plage réaliste. Un seuil maximal de 132\$ par mile a été retenu afin d’éliminer les situations aberrantes.
Cohérence géographique – distance	La distance « à vol d’oiseau » (formule de Haversine) entre les coordonnées de départ et d’arrivée doit être inférieure ou égale à la distance réelle déclarée (<code>trip_distance_miles</code>), à une tolérance près. Un trajet routier ne peut être plus court que la distance géodésique minimale.

TABLE 3 – Règles de cohérence inter-attributs

Calcul du score de cohérence Le score de cohérence pondéré est calculé selon la formule suivante :

$$\text{Score}_{\text{consistency}} = \left(\frac{n_{\text{cohérentes}}}{n_{\text{évaluées}}} \right)^{\text{poids}} \quad (4)$$

5.1.4 Dimension 4 : Unicité (Uniqueness)

L’unicité mesure l’absence de doublons dans le jeu de données. Chaque ligne étant censée représenter une course unique, la présence d’enregistrements dupliqués peut fausser les analyses en surestimant les volumes, les revenus ou les durées moyennes.

Dans le contexte de ce projet, chaque ligne du dataset est supposée correspondre à une course unique. L’unicité est donc évaluée à l’échelle des enregistrements complets, et non sur des clés techniques inexistantes dans le dataset.

Principe d’évaluation Le score d’unicité repose sur le ratio entre le nombre de lignes distinctes et le nombre total de lignes du jeu de données :

$$\text{Score}_{\text{uniqueness}} = \frac{n_{\text{distinctes}}}{n_{\text{totales}}} \quad (5)$$

où $n_{\text{distinctes}}$ représente le nombre de lignes distinctes et n_{totales} le nombre total de lignes dans le dataset.

Pondération par scénario métier Bien que l’unicité soit une dimension globale du dataset, son impact métier varie selon le scénario. Par exemple :

- Pour le scénario Driver, des doublons peuvent artificiellement gonfler les revenus ou les volumes de courses

- Pour le scénario Passenger, ils peuvent biaiser les estimations de prix ou de durée issues de modèles statistiques

Ainsi, le score d'unicité est pondéré par l'importance métier de chaque colonne, selon le scénario considéré, via une élévation à la puissance du poids associé :

$$\text{Score}_{\text{uniqueness}} = \left(\frac{n_{\text{distinctes}}}{n_{\text{totales}}} \right)^{\text{poids}} \quad (6)$$

où w représente le poids métier associé à la colonne pour le scénario considéré.

Cette pondération permet de renforcer l'impact des doublons lorsque les colonnes critiques du scénario sont concernées.

5.2 Assessment

Après avoir défini les dimensions de qualité retenues (complétude, exactitude, cohérence et unicité) ainsi que les méthodes de calcul associées, nous avons procédé à l'évaluation des scores de qualité pour chacun des deux scénarios métier : Driver et Passenger.

	driver_accuracy	passenger_accuracy	driver_completeness	passenger_completeness	driver_consistency	passenger_consistency	driver_uniqueness	passenger_uniqueness
pickup_datetime	0.88	0.88	1.000000	1.000000	0.88	0.88	0.64	0.64
dropoff_datetime	1.00	1.00	1.000000	1.000000	0.88	0.87	0.64	0.61
pickup_latitude	0.78	0.82	1.000000	1.000000	0.95	0.96	0.61	0.68
pickup_longitude	0.78	0.82	1.000000	1.000000	0.95	0.96	0.61	0.68
dropoff_latitude	0.76	0.81	1.000000	1.000000	0.95	0.96	0.61	0.68
dropoff_longitude	0.76	0.81	1.000000	1.000000	0.95	0.96	0.61	0.68
trip_distance_miles	0.96	0.92	0.887508	0.787671	1.00	1.00	0.76	0.57
fare_amount	0.92	0.92	0.759593	0.759593	1.00	1.00	0.57	0.57
passenger_count	1.00	1.00	1.000000	1.000000	NaN	NaN	0.95	0.95
payment_type	1.00	1.00	1.000000	1.000000	NaN	NaN	0.95	0.95

FIGURE 1 – Scores de qualité par dimension pour les scénarios Driver et Passenger

5.3 Vulnérabilités spécifiques par scénario

5.3.1 Scénario Driver

Point critique : Coordonnées GPS Les coordonnées GPS présentent un score d'exactitude de 0.76–0.78 (contre 0.81–0.82 pour le scénario Passenger). Cette vulnérabilité s'explique par le poids élevé (9) attribué à ces variables pour l'identification des zones rentables.

Impact métier : 22–24% d'erreur dans l'analyse géographique, conduisant à un mauvais positionnement des chauffeurs sur le terrain.

5.3.2 Scénario Passenger

Point critique : Complétude de la distance La variable `trip_distance_miles` affiche un score de complétude de 0.79 (contre 0.89 pour le scénario Driver). Cette faiblesse résulte du poids maximal (10) attribué à cette variable, essentielle pour l'estimation tarifaire.

Impact métier : 21% des trajets sans distance affichée, rendant impossible la validation du tarif proposé aux utilisateurs.

5.3.3 Vulnérabilités communes

Variable fare_amount Le montant du tarif présente un score de complétude de 0.76 dans les deux scénarios (poids 10 identique). Cette lacune entraîne une perte de 24% en exploitabilité pour les calculs de revenus (Driver) et les estimations tarifaires (Passenger).

Variable pickup_datetime La date de prise en charge affiche un score de cohérence de 0.88, impacté par les 13 dates injectées en 2025. Cette anomalie génère 12% d'erreur dans les analyses de patterns horaires et la planification des shifts.

5.3.4 Problème d'unicité détecté

Des scores d'unicité particulièrement faibles ont été observés sur plusieurs variables clés :

- `trip_distance_miles` : Score d'unicité de 0.76 (Driver) et 0.57 (Passenger)
- `fare_amount` : Score d'unicité de 0.57 dans les deux scénarios

Interprétation Ces scores faibles suggèrent la présence significative de doublons ou de valeurs répétitives dans le dataset.

Impact métier Driver Pour l'identification des zones de concentration (*hotspots*), ces doublons créent des faux positifs. Certaines zones semblent artificiellement très demandées alors qu'il s'agit de courses dupliquées, faussant l'analyse de la demande géographique.

Impact métier Passenger Les doublons peuvent fausser les statistiques de tarifs moyens et créer des estimations biaisées pour les utilisateurs cherchant à prévoir le coût de leur trajet.

5.3.5 Note méthodologique sur la cohérence

Les variables `passenger_count` et `payment_type` affichent des scores de cohérence égaux à NaN (non applicable). Cette absence de score s'explique par le fait que nous n'avons pas identifié de règles de cohérence inter-colonnes pertinentes pour ces variables dans le contexte de nos scénarios métier. Ces colonnes catégorielles (nombre de passagers, mode de paiement) n'ont pas de relations logiques obligatoires avec les autres variables du dataset qui justifieraient une validation de cohérence.

5.4 Conclusion

Les deux scénarios se situent dans une zone de qualité **MOYENNE** : le dataset reste exploitable mais nécessite des corrections ciblées avant toute utilisation opérationnelle. Les priorités de correction diffèrent selon le cas d'usage :

Scénario Driver

1. **Priorité 1** : Coordonnées GPS (accuracy 0.76–0.78)
2. **Priorité 2** : Déduplication des courses (uniqueness 0.57–0.76)

Scénario Passenger

1. **Priorité 1** : `trip_distance_miles` (completeness 0.79)
2. **Priorité 2** : `fare_amount` (completeness 0.76)
3. **Priorité 3** : Déduplication (uniqueness 0.57)

Cette analyse démontre qu’une même erreur technique peut avoir un impact métier radicalement différent selon le contexte d’usage : les 20 valeurs manquantes de `trip_distance_miles` représentent un défaut mineur pour le Driver (score 0.89) mais majeur pour le Passenger (score 0.79). De même, les problèmes d’unicité affectent davantage le Driver dans son analyse de zones de demande que le Passenger dans ses estimations tarifaires.

6 Data Quality Improvement

Suite à l’évaluation, nous avons développé un pipeline de correction automatisé suivant un principe directeur : corriger systématiquement les erreurs tout en préservant l’intégrité sémantique des données.

6.1 Stratégie Globale de Correction

Notre méthodologie de correction suit un pipeline séquentiel en quatre phases, conçu pour maximiser l’efficacité et éviter les corrections contradictoires :

6.1.1 Phase 1 : Normalisation Syntaxique

Objectif : Standardiser les formats de données pour assurer leur exploitabilité.
Cette phase traite les erreurs de format sans modifier la sémantique des valeurs.

6.1.2 Phase 2 : Déduplication

Objectif : Éliminer les enregistrements redondants.
Exécutée après la normalisation pour détecter les doublons même avec des formats différents.

6.1.3 Phase 3 : Imputation des Valeurs Manquantes

Objectif : Restaurer les valeurs absentes par des méthodes statistiques.
Nécessite des données normalisées et déduplicées pour garantir la fiabilité des modèles.

6.1.4 Phase 4 : Validation de Cohérence

Objectif : Résoudre les incohérences inter-attributs.
Phase finale car elle requiert des données complètes pour calculer les relations entre colonnes.
Cette séquentialité garantit que chaque correction dispose des prérequis nécessaires et que le pipeline est réexécutable sans effets secondaires.

6.2 Corrections Mises en Œuvre

6.2.1 Normalisation des Formats de Dates

Les champs `pickup_datetime` et `dropoff_datetime` présentaient des formats non-standard. Ces formats empêchaient le parsing automatique et rendaient impossibles les calculs temporels.

Approche de correction Nous avons développé un analyseur multi-format capable de :

- Détecter automatiquement le pattern de chaque valeur via expressions régulières
- Convertir les mois textuels français en numéros (janvier → 01, février → 02, etc.)
- Standardiser vers le format ISO 8601 : YYYY-MM-DD HH :MM :SS

6.2.2 Normalisation des Montants Monétaires

Les valeurs de `fare_amount` peuvent contenir des symboles de devises (\$, £, €, ¥), rendant les conversions numériques impossibles.

Approche de correction Extraction de la valeur numérique pure en :

- Supprimant tous les symboles monétaires et caractères non-numériques
- Conservant uniquement les chiffres et le séparateur décimal

6.2.3 Normalisation des Distances

Les valeurs `trip_distance_miles` présentaient des annotations d’unités non-standard (« 2.1 mi »).

Approche de correction Standardisation et extraction des valeurs numériques uniquement, en supprimant les symboles d’unités.

6.2.4 Élimination des Doublons

Le dataset peut contenir plusieurs lignes dupliquées.

Approche de correction Déduplication stricte basée sur l’identité complète des enregistrements (toutes colonnes identiques).

6.2.5 Imputation des Valeurs Manquantes

Les valeurs manquantes dans `trip_distance_miles` et `fare_amount` (attributs critiques pour les deux scénarios) ont été imputées par régression linéaire supervisée, puisque le data profiling a révélé que ces deux attributs sont fortement corrélés.

Imputation des distances manquantes Méthode : Apprentissage supervisé sur données de référence.

Nous avons entraîné un modèle de régression linéaire sur le dataset original (données saines) pour apprendre la relation entre `fare_amount` et `trip_distance_miles`.

Application : Pour chaque enregistrement où la distance est manquante ou négative mais le tarif disponible, le modèle prédit la distance probable.

Imputation des tarifs manquants Méthode : Régression linéaire inverse.

Le même principe est appliqué dans le sens inverse pour prédire `fare_amount` à partir de `trip_distance_miles`.

Application : Imputation des tarifs manquants lorsque la distance est disponible.

6.2.6 Correction des Coordonnées GPS Erronées

Les coordonnées GPS (`pickup_latitude`, `pickup_longitude`, `dropoff_latitude`, `dropoff_longitude`) contiennent des valeurs aberrantes situées hors de la zone géographique attendue (New York).

La correction repose sur un principe clé : si l'un des deux points du trajet (`pickup` ou `dropoff`) est valide et que la distance est fiable, alors la position de l'autre point peut être reconstruite de manière géométriquement cohérente.

Détection des trajets problématiques Un trajet est considéré comme problématique lorsque :

- La distance est valide
- Mais l'un des deux points (`pickup` ou `dropoff`) est situé en dehors de la zone géographique de New York

Approche de correction Lorsqu'un seul des deux points est erroné, le système reconstruit une position plausible en respectant deux contraintes fondamentales :

- La distance du trajet doit rester identique
- Le point corrigé doit se situer à l'intérieur de New York

Pour cela, la correction suit une stratégie hiérarchique :

Préserver la direction du trajet si possible Si une coordonnée erronée existe, sa direction par rapport au point valide est utilisée pour tenter de repositionner le point à la bonne distance.

Recentrage vers le cœur de la ville Si la direction originale ne mène pas à une position valide, une direction vers un centre géographique de New York est testée.

Exploration contrôlée de l'espace En dernier recours, plusieurs directions standards (nord, sud, est, ouest, etc.) sont explorées afin de trouver une position respectant à la fois la distance et la contrainte géographique.

Une correction est acceptée uniquement si une position valide dans New York est trouvée. Les trajets où les deux points sont invalides ou où la distance est erronée sont volontairement ignorés pour éviter toute hallucination géographique.

6.2.7 Correction des Incohérences Temporelles

Les champs temporels (`pickup_datetime`, `dropoff_datetime`) présentent parfois des incohérences : instants manquants d'un côté seulement, ou cas où la date d'arrivée est antérieure à la date de départ.

Principe général La correction repose sur deux idées simples et conservatrices :

- Utiliser les instants valides quand ils existent
- Reconstruire l'instant manquant ou corriger l'incohérence en se basant sur une vitesse de référence calculée à partir des trajets sains

On n'invente pas de données lorsque la distance est invalide ou que les deux timestamps sont manquants.

Détection des erreurs Marquage des enregistrements en erreur lorsque :

- `dropoff < pickup` (arrivée avant départ), ou
- Exactement une des deux dates est nulle (une date manquante mais pas les deux)

Approche de correction

Construire un référentiel de vitesses À partir des trajets jugés sains (pas d'erreur temporelle et distance positive), on calcule la durée observée et la vitesse en miles/heure, puis on agrège ces vitesses par fenêtre temporelle (médiane horaire). Ce référentiel horaire sert à estimer une durée plausible en fonction de l'heure du trajet.

Imputation guidée par la distance et la vitesse de référence Pour chaque enregistrement erroné où la distance est positive :

- Si `dropoff` existe mais `pickup` manque → estimer la durée via la vitesse de référence associée à l'heure de `dropoff`, et déduire `pickup` en soustrayant la durée estimée
- Si `pickup` existe mais `dropoff` manque → estimer la durée via la vitesse de référence associée à l'heure de `pickup`, et déduire `dropoff` en ajoutant la durée estimée
- Si les deux existent mais `dropoff < pickup` → recalculer `pickup` en conservant `dropoff` et en utilisant la durée estimée à partir de la vitesse de référence

Une vitesse par défaut (valeur de secours) est utilisée si le référentiel horaire n'est pas disponible pour l'heure considérée.

6.2.8 Imputation des Attributs Catégoriels et Discrets

Les variables `payment_type` et `passenger_count` présentent des valeurs manquantes.

Approche de correction Nous avons appliqué une imputation par la valeur la plus fréquente (mode) :

- Pour `payment_type`, les valeurs manquantes sont remplacées par le type de paiement le plus courant dans le dataset
- Pour `passenger_count`, les valeurs manquantes sont remplacées par le nombre de passagers le plus fréquent

Cette stratégie repose sur l'hypothèse que les valeurs manquantes suivent la même distribution que les données observées, ce qui permet de préserver la structure statistique globale du dataset sans introduire de biais artificiel important.

6.3 Résultat de la Correction

Une fois que nous avons défini les critères de corrections à effectuer, nous avons appliqué notre pipeline aux données erronées, puis réalisé l'assessment une nouvelle fois pour évaluer l'effet de l'amélioration. Voici les résultats obtenus :

	driver_accuracy	passenger_accuracy	driver_completeness	passenger_completeness	driver_consistency	passenger_consistency	driver_uniqueness	passenger_uniqueness
pickup_datetime	1.00	1.00	1.0	1.0	1.00	1.00	1.0	1.0
dropoff_datetime	1.00	1.00	1.0	1.0	1.00	1.00	1.0	1.0
pickup_latitude	0.89	0.92	1.0	1.0	0.97	0.97	1.0	1.0
pickup_longitude	0.89	0.92	1.0	1.0	0.97	0.97	1.0	1.0
dropoff_latitude	0.95	0.96	1.0	1.0	0.97	0.97	1.0	1.0
dropoff_longitude	0.95	0.96	1.0	1.0	0.97	0.97	1.0	1.0
trip_distance_miles	1.00	1.00	1.0	1.0	1.00	1.00	1.0	1.0
fare_amount	1.00	1.00	1.0	1.0	1.00	1.00	1.0	1.0
passenger_count	1.00	1.00	1.0	1.0	NaN	NaN	1.0	1.0
payment_type	1.00	1.00	1.0	1.0	NaN	NaN	1.0	1.0

FIGURE 2 – Scores après l'amélioration par dimension pour les deux scénarios

En analysant ces résultats, on constate une amélioration considérable sur l'ensemble des dimensions et sur l'ensemble des attributs, où l'on obtient un score parfait de 1 dans la majorité des cas, sauf pour les coordonnées GPS. Ce résultat peut s'expliquer par le fait que certains trajets présentent deux points GPS non valides simultanément, ou que la variable `trip_distance_miles` est elle-même invalide, rendant impossible la reconstruction géométrique des coordonnées. Dans ces deux cas, la correction des coordonnées GPS n'est pas réalisable. Hormis cette limitation technique, notre pipeline a démontré une très haute performance.

7 Échange Inter-Groupes

Lors de la phase d'échange du dataset et du data profiling, plusieurs anomalies ont été identifiées, affectant différentes dimensions de la qualité des données :

- 442 valeurs manquantes, réparties de manière relativement uniforme sur l'ensemble des colonnes
- Variables temporelles (`pickup_datetime`, `dropoff_datetime`) : Plusieurs formats de date hétérogènes ont été détectés, par exemple : « 11-01-2023 20 heure 24 minutes », « 6 janvier 2023 à 9h08 », « 19-01-2023 21h07 »
- Distance du trajet (`trip_distance_miles`) : La colonne contenait des valeurs mixtes combinant nombres et unités (« 11Miles », « 3.44 Miles », « 1.91 MI », « 2.42 mi »)
- Montant de la course (`fare_amount`) : Des symboles de devises variés ont été détectés (£, €, ¥, \$), rendant les valeurs non exploitables directement
- Type de paiement (`payment_type`) : Une forte hétérogénéité des libellés a été observée (Credit card, CB, cb, CREDIT CARD, etc.)

7.1 Résultat de l'Assessment

	driver_accuracy	passenger_accuracy	driver_completeness	passenger_completeness	driver_consistency	passenger_consistency	driver_uniqueness	passenger_uniqueness
pickup_datetime	0.58	0.58	0.629297	0.629297	0.93	0.93	1.0	1.0
dropoff_datetime	0.58	0.54	0.609569	0.572995	0.93	0.92	1.0	1.0
pickup_latitude	0.49	0.57	0.593899	0.666805	1.00	1.00	1.0	1.0
pickup_longitude	0.46	0.55	0.630249	0.698337	1.00	1.00	1.0	1.0
dropoff_latitude	0.56	0.63	0.622825	0.691931	1.00	1.00	1.0	1.0
dropoff_longitude	0.54	0.62	0.637752	0.704795	1.00	1.00	1.0	1.0
trip_distance_miles	0.80	0.63	0.778885	0.606662	0.63	0.39	1.0	1.0
fare_amount	0.37	0.37	0.567960	0.567960	0.39	0.39	1.0	1.0
passenger_count	0.96	0.96	0.925000	0.925000	NaN	NaN	1.0	1.0
payment_type	1.00	1.00	0.948750	0.948750	NaN	NaN	1.0	1.0

FIGURE 3 – Scores du dataset échangé

Le dataset reçu présente des défauts de qualité significatifs sur les dimensions de complétude, exactitude et cohérence, à l'exception de l'unicité où aucun doublon n'a été détecté. Les anomalies identifiées (442 valeurs manquantes, formats hétérogènes, symboles de devises variés, hétérogénéité des libellés) impactent directement les scores et nécessitent l'application d'un pipeline de correction avant toute exploitation opérationnelle.

tex

7.2 Résultat de l'Improvement

Lors de la phase d'improvement, on remarque une nette amélioration des différents scores pour les deux scénarios.

	driver_accuracy	passenger_accuracy	driver_completeness	passenger_completeness	driver_consistency	passenger_consistency	driver_uniqueness	passenger_uniqueness
pickup_datetime	1.00	1.00	0.970391	0.970391	1.0	1.0	1.0	1.0
dropoff_datetime	1.00	1.00	0.970391	0.966752	1.0	1.0	1.0	1.0
pickup_latitude	0.82	0.86	0.796236	0.837592	1.0	1.0	1.0	1.0
pickup_longitude	0.72	0.78	0.903189	0.923859	1.0	1.0	1.0	1.0
dropoff_latitude	0.80	0.84	0.882842	0.907631	1.0	1.0	1.0	1.0
dropoff_longitude	0.85	0.88	0.833748	0.868126	1.0	1.0	1.0	1.0
trip_distance_miles	1.00	1.00	0.993766	0.987570	1.0	1.0	1.0	1.0
fare_amount	1.00	1.00	0.987570	0.987570	1.0	1.0	1.0	1.0
passenger_count	0.96	0.96	1.000000	1.000000	NaN	NaN	1.0	1.0
payment_type	1.00	1.00	1.000000	1.000000	NaN	NaN	1.0	1.0

FIGURE 4 – Scores du dataset reçu après l'improvement

7.3 Erreurs Non Détectées

Malgré l'efficacité globale du pipeline de correction, certaines anomalies introduites dans le dataset n'ont pas été détectées par notre système d'évaluation :

- **Transformation de chiffres en version écrite** : Pour la variable `passenger_count`, certaines valeurs numériques ont été transformées en leur équivalent textuel (ex : 1 devient « un » ou « Un », 2 devient « deux », etc.). Cette anomalie n'a pas été détectée car notre pipeline ne prévoyait pas de validation spécifique sur la nature numérique stricte de cette variable catégorielle.
- **Espaces parasites** : Des espaces ont été ajoutés avant et après les valeurs de toutes les variables sur certains tuples sélectionnés aléatoirement (ex : « _2.5_ » au lieu de « 2.5 »). Ces espaces invisibles peuvent perturber les comparaisons de chaînes et les conversions de types, mais n'ont pas été traités par notre phase de normalisation syntaxique.

Impact métier Ces erreurs non détectées peuvent avoir des conséquences opérationnelles :

- Les valeurs textuelles de `passenger_count` empêchent les calculs statistiques et les agrégations numériques, faussant potentiellement les analyses de capacité des véhicules.
- Les espaces parasites peuvent provoquer des échecs silencieux lors des conversions numériques, générant des valeurs manquantes non identifiées ou des erreurs dans les jointures et les comparaisons.

7.4 Limites du Pipeline de Detection et de Correction

Bien que notre pipeline soit performant en matière de normalisation des formats, d'imputation des valeurs manquantes et de validation de cohérence, il présente néanmoins des limitations importantes et ne couvre pas l'ensemble des anomalies potentielles.

Erreurs non détectées Les erreurs non identifiées dans le dataset de l'autre groupe (transformation de chiffres en texte pour `passenger_count`, espaces parasites) illustrent ces limites. De nombreuses autres anomalies subtiles peuvent échapper à notre système de validation.

Exemple d'incohérence temporelle non détectée Considérons le cas suivant : un trajet avec `pickup_datetime` = « 12 janvier 2023 à 20h00 » et `dropoff_datetime` = « 12 janvier 2024 à 20h30 ». Cette incohérence ne serait pas détectée par notre règle actuelle de cohérence temporelle, car elle respecte formellement la contrainte `pickup_datetime < dropoff_datetime`. Pourtant, un trajet d'une durée d'un an est manifestement aberrant pour un taxi urbain.

Pistes d'amélioration Pour renforcer la robustesse du pipeline, des règles de validation additionnelles pourraient être implémentées :

- **Contrainte d'année et de mois** : Imposer que `pickup_datetime` et `dropoff_datetime` appartiennent à la même année et au même mois, reflétant la réalité opérationnelle des courses de taxi.
- **Contrainte de durée maximale** : Limiter la durée maximale d'un trajet à une valeur plausible (par exemple, 3 heures), au-delà de laquelle l'enregistrement serait considéré comme incohérent.
- **Validation croisée temporelle-géographique** : Vérifier la cohérence entre la durée du trajet, la distance parcourue et les coordonnées GPS pour détecter des combinaisons physiquement impossibles.

Ces limitations soulignent la nécessité d'une approche itérative et évolutive dans la conception de pipelines de qualité des données. Chaque nouveau cas d'usage ou anomalie détectée doit alimenter l'enrichissement progressif des règles de validation, dans une logique d'amélioration continue.

8 Conclusion

Ce projet nous a permis d’explorer en profondeur la thématique de la qualité des données, révélant une complexité bien supérieure à ce qu’elle peut initialement paraître. Au-delà des aspects techniques, ce travail s’est avéré particulièrement riche et formateur, nécessitant une approche créative et méthodique à chaque étape.

L’élaboration de notre pipeline de correction a constitué le cœur de ce projet. Bien que les résultats obtenus soient satisfaisants, avec des améliorations significatives observées sur l’ensemble des dimensions de qualité, nous sommes conscients des limitations de notre approche. Les erreurs non détectées lors de l’échange inter-groupes, ainsi que les cas limites identifiés dans l’analyse, démontrent qu’il est difficile, voire impossible, de concevoir dès le départ un pipeline parfaitement exhaustif.

Cette expérience nous a enseigné qu’un système de qualité des données ne peut être considéré comme un produit fini, mais doit plutôt être envisagé comme un processus évolutif. La qualité des données s’inscrit dans une démarche d’amélioration continue, où chaque anomalie détectée, chaque cas d’usage exploré, et chaque retour d’expérience enrichissent progressivement les règles de validation et les mécanismes de correction.

L’approche par scénarios métier (Driver et Passenger) a par ailleurs démontré l’importance du contexte dans l’évaluation de la qualité : une même erreur technique peut avoir des impacts radicalement différents selon le cas d’usage considéré. Cette notion de *fitness for use* constitue un enseignement majeur de ce projet et devra guider toute démarche future de gouvernance des données.

En définitive, ce travail illustre que la qualité des données n’est pas un état binaire (bon ou mauvais) mais un équilibre dynamique entre exigences métier, contraintes techniques et amélioration progressive. Les compétences développées dans ce projet — conception de règles de validation, imputation intelligente, correction géométrique, analyse contextuelle — constituent des fondations solides pour aborder les défis de qualité des données dans des contextes professionnels réels.

Procédure d’installation et d’utilisation du code

— Fichiers du projet :

- `Taxi modifié.csv` : Fichier CSV corrompu par nos erreurs injectées.
- `df corrupted.csv` : Fichier CSV reçu de l’autre groupe.
- `profiling.ipynb` : Vue générale des données (valeurs manquantes, doublons, distributions)
- `assessment.ipynb` : Évaluation de la qualité selon les dimensions (complétude, précision, cohérence, unicité)
Nécessite les fichiers de **coordonnées GPS NYC** et de **cartes de l’eau** pour la cohérence géographique
- `improvement.ipynb` : Application des stratégies de correction et amélioration de la *fitness for use*

— Utilisation :

- Ouvrir chaque notebook dans **Jupyter Notebook** ou **Google Colab**
- Exécuter les cellules dans l’ordre
- Visualiser les scores et résultats directement dans le notebook

Garder tous les fichiers nécessaires dans le **même dossier** pour éviter les erreurs de chemin

Références

- [1] Mohammed, S., Ehrlinger, L., Harmouch, H., Naumann, F., & Srivastava, D. (2024). *The Five Facets of Data Quality Assessment*. arXiv preprint arXiv :2403.00526.
- [2] Heinrich, B., Hristova, D., Klier, M., Schiller, A., & Szubartowicz, M. (2018). *Requirements for Data Quality Metrics*. ACM Journal of Data and Information Quality, 9(2), Article 12, 1–32.
- [3] Dong, S., Sahri, S., & Palpanas, T. (2024). *Data Quality Awareness : A Journey from Traditional Data Management to Data Science Systems*. arXiv preprint arXiv :2411.03007.
- [4] Pipino, L. L., Lee, Y. W., & Wang, R. Y. (2002). *Data Quality Assessment*. Communications of the ACM, 45(4), 211–218.
- [5] Ehrlinger, L., Werth, B., & Wöß, W. (2018). *Automated Continuous Data Quality Measurement with QuaIIe*. International Journal on Advances in Software, 11(3–4), 400–417.